

Informe sobre IA y métricas Android

Auto Robot

Campo	Detalle
Materia	Sistemas Operativos Avanzados
Comisión	Martes
Grupo	M4
Integrantes y DNI	COARITE COARITE, Ivan Enrique (94447552); GALVEZ, Ezequiel Maximiliano (37659307); GRAMAJO, Héctor Marcelo (34519918); NOVOA ROMERO, Martín Ricardo (43087221); RIOS DI GAETA, Bautista (46431397)
Fecha	06/07/2026

Repositorio GitHub: <https://github.com/UNLAM-SOA/2026-SOA-Q1-M4>

1. Herramienta utilizada

Se utilizó OpenAI Codex como asistente de programación para preparar, revisar y documentar la versión Android con vibecoding del proyecto Auto Robot.

2. Modelo de IA utilizado

Modelo: OpenAI GPT-5 en entorno Codex.

3. Prompt técnico utilizado

El prompt se estructuró con el formato técnico indicado por la cátedra y con el mismo criterio usado por otros grupos de la comisión martes: contexto, stack tecnológico, funcionalidades, requisitos concretos, restricciones y medición final.

El prompt completo queda documentado en Informes/Prompt_Tecnico_Android.md.

Resumen del prompt técnico:

LENGUAJE DE PROGRAMACION:

- Kotlin, XML y Gradle Kotlin DSL.

PLATAFORMA:

- Aplicacion Android nativa para smartphone conectada a un ESP32 Auto Robot por MQTT.

FUNCIONALIDADES A IMPLEMENTAR:

- Control manual del robot con comandos forward, backward, left, right y stop.
- Control por sensor de rotacion del smartphone.
- Pantalla de telemetria con distancia izquierda, distancia derecha, luz y estado.
- Configuracion de broker MQTT desde la app.
- Version manual y version con vibecoding para comparar metricas.

REQUISITOS ESPECIFICOS:

- Usar `SensorManager`, `SensorEventListener` y `TYPE_ROTATION_VECTOR`.
- Publicar comandos en `robot/command`.
- Suscribirse a `robot/sensors` y `robot/state`.
- Interpretar `robot/sensors` como `distanciaIzquierda;distanciaDerecha;luz`.
- Medir CPU y memoria Android con `get_cpu_mem_smarthpone.sh`.

RESTRICCIONES:

- No bloquear el hilo principal.
- No cambiar los topics MQTT.
- No hardcodear credenciales finales.
- Medir reposo y accion con sensor Android durante 10 segundos.

4. Refinamientos realizados

Durante la generación y adaptación se pidió que la versión con vibecoding:

- Mantenga el mismo contrato MQTT que la versión manual.
- Use comandos compatibles con el ESP32: `forward`, `backward`, `left`, `right`, `stop`.
- Conserve la pantalla de control manual y la pantalla de sensores.
- Use un `applicationId` diferente para poder instalar ambas APK en el mismo teléfono.
- Permita ejecutar las métricas Android con el script oficial de la cátedra.
- Mantenga el ciclo de vida correcto del sensor Android: registro en `onResume()` y liberación en `onPause()`.
- No publique comandos repetidos si la dirección detectada no cambia.

5. Cantidad de tokens de IA utilizados

La herramienta utilizada fue Codex. Como no expone un contador total exportable para la sesión completa, se deja un valor estimado a partir del volumen de interacciones, análisis de código, generación de documentación, ajustes y validaciones realizadas.

Valor estimado registrado: **180.000 tokens**.

6. Tiempo para obtener una versión funcional

Versión Android	Tiempo de desarrollo	Observaciones
Manual	Aprox. 10 h	Código base del repo en <code>Android/Proyecto_sin_Vibecoding</code> , desarrollado manualmente por el grupo.
Vibecoding total	Aprox. 4 h	Código preparado en <code>Android/Proyecto_con_Vibecoding</code> usando prompt técnico, refinamientos, compilación, documentación y mediciones.

7. Métricas CPU y memoria

Las métricas se realizan sobre Android, no sobre el ESP32. Se utiliza el script oficial enlazado por la cátedra:

```
Informes/Scripts/get_cpu_mem_smartphone.sh
```

Link de origen: <https://gitlab.com/so-unlam/Material-SOA/-/tree/master/Ejemplos%20Android/UsoCpuMemAndroid>

Paquetes a medir:

Versión	Paquete Android
Manual	dev.mnvoa.SOA
Vibecoding	dev.mnvoa.SOA.vibecoding

Casos solicitados por la clase:

Caso	Versión	Situación	Acción	Duración
1	Manual	Reposo	Abrir la app y no interactuar	10 s
2	Manual	Acción con sensor	Abrir pantalla de sensores y mover/inclinar el teléfono	10 s
3	Vibecoding	Reposo	Abrir la app y no interactuar	10 s
4	Vibecoding	Acción con sensor	Abrir pantalla de sensores y mover/inclinar el teléfono	10 s

Estado actual de las mediciones

Las mediciones reales fueron completadas el 07/07/2026 en un Samsung SM-A325M con Android 13, 8 núcleos y 3645.54 MB de RAM total. Ambas APK fueron instaladas por ADB en el mismo teléfono. Se utilizó la columna 9 de CPU, validada previamente con adb shell top.

Caso	PSS MB	RAM usada %	CPU promedio %	Observación
Manual reposo	180.68	4.9563	241.00	Sin interacción.
Manual acción sensor	190.00	5.2118	62.50	Control por TYPE_ROTATION_VECTOR.
Vibecoding reposo	204.00	5.5958	0.00	Sin interacción.
Vibecoding acción sensor	197.39	5.4145	0.00	Control por TYPE_ROTATION_VECTOR.

En esta corrida, la versión manual tuvo menor PSS en reposo, pero mayor consumo de CPU. La versión con vibecoding usó algo más de memoria, aunque se mantuvo estable en CPU durante las dos mediciones.

8. Procedimiento de medición

- Instalar la APK manual en el teléfono.

- Instalar la APK vibecoding en el teléfono.
- Conectar el teléfono por USB con depuración habilitada.

Copiar el script:

```
adb push Informes/Scripts/get_cpu_mem_smarthpone.sh
/data/local/tmp/get_cpu_mem_smarthpone.sh
adb shell chmod 755 /data/local/tmp/get_cpu_mem_smarthpone.sh
```

Verificar la columna de CPU:

```
adb shell top
```

En la clase se usa como referencia la columna 9. En el teléfono medido se confirmó la misma columna con adb shell top.

Ejecutar medición manual:

```
adb shell sh /data/local/tmp/get_cpu_mem_smarthpone.sh dev.mnovoa.SOA 9
```

Ejecutar medición vibecoding:

```
adb shell sh /data/local/tmp/get_cpu_mem_smarthpone.sh dev.mnovoa.SOA.vibecoding 9
```

Repetir cada comando en reposo y con acción de sensor. En la acción de sensor se debe abrir SensorActivity e inclinar el teléfono durante la ventana de medición.

9. Código en Git

Estructura solicitada:

```
Android/
  Proyecto_sin_Vibecoding/
  Proyecto_con_Vibecoding/
Embebido/
  Proyecto_sin_Vibecoding/
  Proyecto_con_Vibecoding/
Informes/
```

Enlace al directorio Android manual: https://github.com/UNLAM-SOA/2026-SOA-Q1-M4/tree/main/Android/Proyecto_sin_Vibecoding

Enlace al directorio Android vibecoding: https://github.com/UNLAM-SOA/2026-SOA-Q1-M4/tree/main/Android/Proyecto_con_Vibecoding

10. Rúbrica de funcionalidad Android

Ítem	Manual	Vibecoding	Observación
Compila	Sí	Sí	Verificado con <code>./gradlew.bat :app:assembleDebug</code> .
Ejecuta en teléfono	Sí	Sí	Instalado y medido en Samsung SM-A325M por ADB.
Ejecución en background	Parcial	Parcial	MQTT y Flow se procesan fuera del flujo directo de UI.

Ítem	Manual	Vibecoding	Observación
Sensores Android	Sí	Sí	Usa TYPE_ROTATION_VECTOR.
Permisos Android	Sí	Sí	Usa permiso INTERNET.
Comunicación con embebido	Sí	Sí	Usa MQTT.
Cumplimiento funcional	Sí	Sí	Control manual, control por sensor y telemetría.
Diseño de app	Sí	Sí	Pantallas de control, sensores y configuración.

11. Conclusión

La actividad de IA queda orientada a Android. La comparación se realizó entre la APK manual y la APK generada/adaptada con vibecoding, midiendo CPU y memoria con el script ADB de la cátedra.

El enfoque con vibecoding acelera la obtención de una versión funcional porque permite generar estructura, pantallas, conexión MQTT y manejo de sensores a partir de un prompt técnico. Sin embargo, requiere supervisión humana para validar que los topics MQTT coincidan con el ESP32, que el sensor Android respete el ciclo de vida de la Activity y que el código no agregue complejidad innecesaria.

Las mediciones finales dependen del modelo del teléfono, versión de Android, RAM disponible, procesos activos y columna de CPU que devuelva top. Por eso se documentó el dispositivo usado y se mantuvo el mismo entorno para las cuatro pruebas.